

Homework 1

(Due date: 4 October 2024 at 11:59pm)

For Q1 and Q2, you need to submit your answer as a PDF file.

For Q3-Q6, you need to submit the folder than includes the code of each question as a zip file.

Question 1 (10%):

- a) Consider the microprocessor-based computer systems discussed in class (Topic 1). Briefly explain the functionality of the address bus, data bus, and control bus.
- The **address bus** identifies the specific memory location or I/O device the processor intends to communicate with. Each device has a unique address, allowing the CPU to send data directly to a specific address. Additionally, it allows for bidirectional communication.
- The **data bus** carries data between the microprocessor and memory or I/O devices. It allows for bidirectional communication as well.
- The **control bus** carries control signals that manage operations between the CPU, memory, and I/O devices. It indicates whether the CPU is reading from or writing to a device and determines the type of operation being performed. It allows for unidirectional communication.
- b) Briefly explain the five steps of the instruction execution cycle (Topic 2 - maximum two lines explanation for each step).
1. The Fetch Instruction step allows the CPU to retrieve the instruction from an area of the memory called the instruction memory.
 2. The Instruction Decode step allows the CPU to decode the fetched instruction into its binary representation that might reveal if the instruction has operands (input values).
 3. The Fetch Operand step allows the CPU to fetch the operands from registers and data memory, that is if operands are involved.
 4. The Execute step allows the CPU to execute the instruction using any operand values it fetched previously, and it updates the status flags accordingly.
 5. The Store Output allows the CPU to store the result of executed instruction in the output operand, that is if an output operand was part of the instruction.

```

.data
; Your data goes here
a BYTE -18 ;byte
b WORD 0F7h ;short int
cc WORD 0DBE7h ;short int
d DWORD -23 ;int

.code
main PROC

    movsx eax, a
L1:    neg eax

    mov eax, 0
    movzx eax, b
L2:    add al, 17h

    mov eax, 0
    movsx eax, cc

L3:    sub al, 0FFh

    mov eax, -5
    sub eax, d
L4:    mov d, eax

    exit
main ENDP
END main

```

Question 2 (20%) - Consider the following program

Show the values of register EAX as well as the status of the ZF, SF, CF, and OF after executing of instructions located at labels L1, L2, L3, and L4.

L1: EAX = 00000012 , ZF = 0, SF = 0, CF = 1, OF = 0

L2: EAX = 0000000E , ZF = 0, SF = 0, CF = 1, OF = 0

L3: EAX = FFFFDBE8 , ZF = 0, SF = 1, CF = 1, OF = 0

L4: EAX = 00000012 , ZF = 0, SF = 0, CF = 0, OF = 0

Question 3 (15%): Write a program to read an integer value **n**. The program should compute and print the value of **A** that is expressed in the following equation:

$$A = \sum_{ii=1}^n (3ii - 5)$$

You need to use LOOP to compute **A**. Below please find a sample of the expect output of the program

```

C:\> Select D:\AUS_one_drive\OneDrive - aus.edu\Cou
Please enter the value of n: 10
The value of A is: +115
Press any key to continue...

```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

```

.data
    str1 BYTE "Please enter the value of n: ", 0
    str2 BYTE "The value of A is: ", 0
    mult DWORD 3
    sum DWORD 0

.code
main PROC
    mov edx, OFFSET str1
    call WriteString

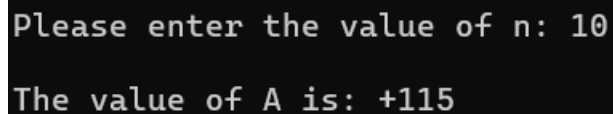
    call ReadInt
    mov ecx, eax

again:
    mov eax, mult
    sub eax, 5
    add sum, eax
    add mult, 3
    loop again

    call crlf
    mov edx, OFFSET str2
    call WriteString
    mov eax, sum
    call WriteInt

    exit
main ENDP
END main

```

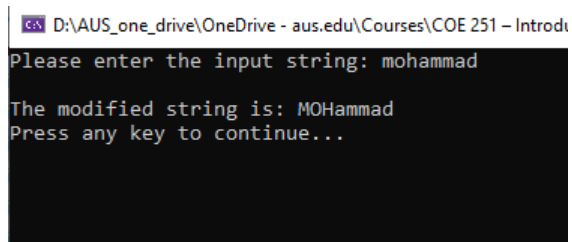


```

Please enter the value of n: 10
The value of A is: +115

```

Question 4 (15%): Write a program that defines a byte array, called **String1**, with a size of 10 bytes. The program should read a string from the keyboard (all characters are small letters, and the maximum number of characters is 9) using the Irvine **ReadString** procedure and store the string in String1. The program should capitalize the first three letters of the string and print the modified string using the Irvine **WriteString**. You need to use the “indirect operand” addressing mode (Topic 4 – part 3) to answer this question. Below please find a sample of the expect output of the program.



```

D:\AUS_one_drive\OneDrive - aus.edu\Courses\COE 251 - Intro
Please enter the input string: mohammad
The modified string is: MOHammad
Press any key to continue...

```

Hint: below is the explanation of the **ReadString** procedure.

ReadString PROC

Reads a string of up to ECX non-null characters from standard input, stopping when the user presses the Enter key.

A null byte is stored following the characters input, but the trailing carriage return and line feed characters are not placed into the buffer.

ECX should always be smaller than the buffer size (never equal to the buffer size) because the null byte could be the (ECX+1)th character stored.

Call args: EDX points to the input buffer
ECX max number of non-null chars to read

Return arg: EAX = size of input string.

Example:

```
.data
MAX = 80                      ;max chars to read
stringIn BYTE MAX+1 DUP (?)  ;room for null

.code
    mov  edx,OFFSET stringIn
    mov  ecx,MAX               ;buffer size - 1
    call ReadString
```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

```
.data
    MAX = 10
    string1 BYTE MAX+1 DUP (?)
    string2 BYTE "Please enter the input string: ", 0
    string3 BYTE "The modified string is: ", 0
```

```
.code
main PROC
    mov edx, OFFSET string2
    call WriteString

    mov edx, OFFSET string1
    mov ecx, MAX
    call ReadString

    mov ecx, 3
    mov esi, OFFSET string1
```

```
again:
    mov al, [esi]
    sub al, 32
    mov [esi], al
    inc esi

    loop again

    call crlf

    mov edx, OFFSET string3
    call WriteString
    mov edx, OFFSET string1
    call WriteString
```

```

        exit
main ENDP
END main

```

```

Please enter the input string: mohammad

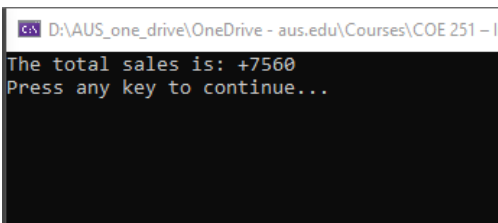
The modified string is: MOHammad

```

Question 5 (20%): Write a program to calculate the total sales of a store in five days. The daily sales of the store in each one of the five days are (all values are decimal): 1050, 1450, 1300, 2010, and 1750. When you write the program, you need to do the following:

1. In the data segment, define an array of word size (two bytes) to store the five daily sales of the store.
2. In the code segment, use the “direct-offset operand” approach (Topic 4-part 1) to compute and display the total sales in five days.

Below please find a snapshot of the expected output.



```

D:\AUS_one_drive\OneDrive - aus.edu\Courses\COE 251 - I
The total sales is: +7560
Press any key to continue...

```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```

    array WORD 1050, 1450, 1300, 2010, 1750
    str1 BYTE "The total sales is: ", 0

```

.code

main PROC

```

    mov edx, OFFSET str1
    call WriteString

```

```

    mov eax, 0

```

```

    add ax, array
    add ax, array+2
    add ax, array+4
    add ax, array+6
    add ax, array+8

```

```

    call WriteInt

```

```

    exit

```

main ENDP

END main

COE 251

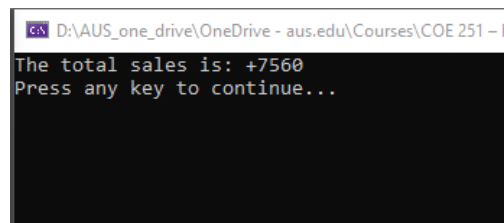
Department of Computer Science and Engineering
Fall 2024

AUS | الجامعة الأميركية في الشارقة
American University of Sharjah

The total sales is: +7560

Question 6 (20%): Modify the program in Question 5 to calculate the total sales of the store in five days using the “indirect operand” approach (Topic 4 – part 3) and LOOPS. You need to use the EBX register to point to the address of the array, and then perform the addition of the five daily sales through the BX register. You also need to use the “\$” directive to determine the size of the array that stores the daily sales.

Below please find a snapshot of the expected output.



```

D:\AUS_one_drive\OneDrive - aus.edu\Courses\COE 251 - I
The total sales is: +7560
Press any key to continue...

```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

```

.data
    array WORD 1050,1450,1300,2010,1750
    len_array = ($ - array)/2
    str1 BYTE "The total sales is: ", 0

```

.code

main PROC

```

    mov edx, OFFSET str1
    call WriteString

```

```

    mov eax, 0
    mov ebx, OFFSET array
    mov ecx, len_array

```

```

    mov ax, 0

```

again:

```

    add ax, [ebx]
    add ebx, 2
    loop again

```

```

    call WriteInt

```

```

    exit

```

main ENDP

END main

The total sales is: +7560